



**UNIDADE UNIVERSITÁRIA DE DOURADOS CURSO DE BACHARELADO
EM CIÊNCIA DA COMPUTAÇÃO DISCIPLINA DE REDES DE COMPUTADORES**

Igor Roberto Michalski de Souza

**Trabalho de Redes de Computadores
Processadora Escalonadora**

Dourados, MS

2024

Sumário

1. INTRODUÇÃO.....	6
1.1 Escalonadora-Processadora.....	6
1.2 Objetivos do Trabalho.....	6
1.3 Programas a Serem Desenvolvidos.....	6
2. DESENVOLVIMENTO.....	8
2.1 Decisões de Implementações.....	8
2.2 Principais Funções.....	9
2.3 Retransmissão das Mensagens.....	14
2.4 Testes e Análise.....	15
REFERÊNCIAS.....	18

1. INTRODUÇÃO

1.1 Escalonadora-Processadora

A computação em nuvem refere-se à disponibilidade sob demanda de recursos computacionais, como armazenamento de dados e capacidade de processamento, acessíveis remotamente pela internet. Este projeto visa praticar a programação com a biblioteca Socket utilizando o protocolo TCP para criar um sistema de computação em nuvem que trata múltiplas conexões.

1.2 Objetivos do Trabalho

O principal objetivo é desenvolver um sistema que permita ao usuário enviar um arquivo fonte para uma máquina escalonadora, que, por sua vez, enviará o arquivo para uma máquina processadora. A máquina processadora compilará, executará e devolverá o resultado ao cliente. Em essência, será implementada uma nuvem para a compilação e execução de programa. Para este projeto, os programas do cliente, portal e servidor serão desenvolvidos em Python, enquanto os arquivos fonte enviados para compilação serão em C.

1.3 Programas a Serem Desenvolvidos

O trabalho consistirá na implementação de três programas: cliente, portal e servidor.

1. Cliente:

- Se conecta ao portal usando o endereço IP e a porta fornecidos como parâmetros.
- Permite o envio de um ou mais arquivos fontes.
- Envia mensagens de envio (S) e listagem (L) ao portal e exibe os resultados na tela.

2. Portal:

- Recebe conexões do cliente e se conecta ao servidor de processamento.
- Implementa dois métodos de seleção do servidor de processamento: aleatório e Round-Robin.
- Encaminha os arquivos para os servidores de processamento e retorna os resultados ao cliente.

3. Servidor (Processador):

- Recebe arquivos fonte do portal.

- Compila, executa os arquivos e retorna os resultados (erros de compilação, resultados corretos ou incorretos) ao portal.

2. DESENVOLVIMENTO

2.1 Decisões de Implementações

Meu código tem como objetivo principal pegar um ou mais arquivos que o cliente deseja enviar e encaminhá-los após serem segmentados. A escolha pela segmentação do arquivo foi feita com base em referências e questões de segurança. Quando o arquivo chega ao portal e a conexão já está estabelecida é criada uma thread, o arquivo é recebido e montado no portal. Essa abordagem de montar o arquivo no portal foi implementada para facilitar os testes, garantindo que todas as etapas estão funcionando corretamente e nenhuma etapa foi pulada.

Após o portal receber o arquivo, ele verifica o modo especificado nos parâmetros de execução e decide se irá escolher um servidor aleatoriamente ou seguir a sequência de Round Robin. O envio e a recepção no servidor seguem o mesmo procedimento. A única diferença é que, no servidor, sempre que uma conexão é recebida, uma thread é criada para processar o arquivo recebido, permitindo que o servidor continue a receber outras conexões.

Por exemplo, ao enviar 4 arquivos usando o Round Robin:

- Arq1 vai para o serv1
- Arq2 vai para o serv2
- Arq3 vai para o serv3
- Arq4 volta para o serv1

Os servidores precisam estar disponíveis para evitar erros ou atrasos no processo.

Depois que o arquivo é executado no servidor, o resultado é retornado por mensagem, passando do servidor para o portal. No portal, usamos o socket de conexão com o cliente, garantindo que o resultado seja enviado para o cliente correto, mesmo com múltiplos clientes conectados ao portal.

2.2 Principais Funções

- Enviar Arquivos

```
def enviar_arquivo(ip, port, nome_arquivo):  
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as envia:  
        envia.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)  
        envia.connect((ip, port))  
  
        #Enviar o nome do arquivo primeiro  
        nome_arquivo_bytes = nome_arquivo.encode('utf-8')  
        envia.send(len(nome_arquivo_bytes).to_bytes(4, 'big'))  
        envia.send(nome_arquivo_bytes)  
  
        #Enviar o conteúdo do arquivo  
        with open(nome_arquivo, "rb") as arquivo:  
            ler_buffer = arquivo.read(1024)  
            while ler_buffer:  
                envia.send(ler_buffer)  
                ler_buffer = arquivo.read(1024)
```

Uma das funções mais usadas no código, se não há mais usada. Essa função tem como objetivo enviar o nosso arquivo .c para o IP e PORTA passadas como parâmetro.

Etapas da função:

1. Cria um socket e se conecta.
2. Separa o tamanho do nome do arquivo e envia o tamanho.
3. Envia o nome do arquivo.
4. Abre o arquivo para leitura.
5. Gera um loop que pega 1024 bytes do arquivo e envia até que o arquivo inteiro seja enviado.

- Receber Arquivos

```
def receber_arquivos(client_socket):
    arquivos_recebidos = []
    try:
        while True:
            #Recebe tamanho e nome do arquivo
            file_name_size = client_socket.recv(4)
            if not file_name_size:
                break
            file_name_size = int.from_bytes(file_name_size, 'big')
            file_name = client_socket.recv(file_name_size).decode('utf-8')

            #Recebe o conteudo do arquivo
            with open(file_name, 'wb') as arquivo:
                while True:
                    ler_buffer = client_socket.recv(1024)
                    if not ler_buffer:
                        break
                    arquivo.write(ler_buffer)

            arquivos_recebidos.append(file_name)

    except Exception as e:
        exit(1)
    finally:
        return arquivos_recebidos
```

A segunda função mais usada com certeza é a que recebe os arquivos enviados pela função anterior. Essa função, como o próprio nome diz, recebe os arquivos .c que foram enviados.

Por parâmetro é passado o socket que foi criado e ouviu a conexão na main.

Etapas da função:

1. É criado uma lista para receber o nome dos arquivos.
2. Primeiro se recebe o tamanho do nome do arquivo convertido para inteiro.
3. Recebemos o nome do arquivo e um arquivo com esse nome é criado no diretório de quem recebe.
4. Gera um loop que recebe 1024 bytes do arquivo e grava até que o arquivo inteiro seja recebido.
5. Por fim, o nome do arquivo é adicionado a lista de arquivos recebidos.

- Tratar Cliente (no portal)

```
def tratarCliente(client_socket, mode, RR):
    #Assim que um cliente conectou iniciamos o tratamento dele
    try:
        #Recebe o arquivo enviado pelo cliente
        arquivos_recebidos = receber_arquivos(client_socket)

        resultado_final = ""

        #Caminhos para o tipo de escolha de processamento
        if mode == "random":
            for arquivo in arquivos_recebidos:
                server = random.choice(servers) #Separa o servidor destino
                enviar_arquivo(server[0], server[1], arquivo) #Envia o Arquivo para o Servidor
                resultado = receber_resultado(server[0], 8001) #Chama a função que irá tratar a resposta do servidor
                resultado_final += resultado + "\n" #Armazena a resposta

            elif mode == "round-robin":
                for arquivo in arquivos_recebidos:
                    server = servers[RR] #Separa o servidor destino
                    enviar_arquivo(server[0], server[1], arquivo) #Envia o Arquivo para o Servidor
                    resultado = receber_resultado(server[0], 8001) #Chama a função que irá tratar a resposta do servidor
                    resultado_final += resultado + "\n" #Armazena a resposta
                    RR = (RR + 1) % len(servers) #Atualiza index do Round Robin

    except Exception as e:
        exit(1)
    finally:
        #Enviar o resultado da execução de volta para o cliente
        enviar_string(client_socket.getpeername()[0], 8002, resultado_final)
        client_socket.close()
    return RR
```

Essa função ela já está bem explicada no código, mas trouxe aqui por ela ser importante. Basicamente, sempre que se tem uma conexão do cliente ela entra em ação.

Etapas da função:

1. Chama a função receber_arquivos.
2. Cria uma variável para armazenar as respostas que o servidor irá retornar.
3. Com base no modo escolhido, ela faz seu caminho, com um loop que roda até que todos os arquivos enviados pelo cliente sejam encaminhados para um servidor.
4. Envia o arquivo para o servidor usando a função enviar_arquivo.
5. Recebe o resultado e concatena com a variável resultado_final.
6. Por fim envia o resultado_fim para o cliente que enviou os arquivos e está aguardando a resposta, fecha a conexão com o cliente e atualiza a variável global de Round e Robin, para requisições futuras seguirem a ordem de servidores.

- Tratamento Portal (no servidor)

```
def tratarPortal(portal_socket):
    try:
        #Recebe o arquivo enviado pelo Portal
        file_name = receber_arquivos(portal_socket)
        if file_name:
            #Executa o arquivo.c e armazena o resultado
            result = Executar(file_name)

            #Enviar o resultado da execução de volta para o portal
            enviar_string(IP_PORTA_OndePortalOuve[0], IP_PORTA_OndePortalOuve[1], result)

    except Exception as e:
        exit(1)
    finally:
        portal_socket.close()
```

Uma função bem simples que controla os passos dentro do servidor.

Etapas da função:

1. Recebe o arquivo que vai ser executado
2. Chama a função que vai executar o arquivo e devolver seu resultado
3. Envia o resultado de volta para o portal utilizando a função `enviar_string`.

- Execução (no servidor)

```
def Executar(file):
    try:
        #Realiza o processo para compilar e rodar o arquivo
        compile_cmd = f"gcc {file} -o {file.split('.')[0]}"
        result_compile = subprocess.run(compile_cmd, shell=True, check=True, capture_output=True) #Cria um sub Processo para compilar o .c

        run_cmd = f"./{file.split('.')[0]}"
        result_run = subprocess.run(run_cmd, shell=True, check=True, capture_output=True) #Cria um sub Processo para Rodar a aplicação .exe

        #Se deu tudo certo retorna o resultado
        return result_run.stdout.decode()

    except subprocess.CalledProcessError as e:
        #Em caso de erro retorna o erro
        return e.stderr.decode()
```

Essa função é responsável por gerar a execução do arquivo .c e retornar seu resultado, sendo ele um erro ou um resultado correto.

Etapas da função:

1. Cria uma string de compilação
2. Gera um subprocesso que compila o arquivo. (.c)
3. Cria uma string que irá ser usada para executar o arquivo compilado.
4. Gera um subprocesso para executar o arquivo. (.exe)
5. Retorna o resultado correto ou o erro.

-

- Enviar String

```
def enviar_string(ip, port, mensagem):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as envia:
        envia.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        envia.connect((ip, port))

        mensagem_bytes = mensagem.encode('utf-8')
        envia.send(len(mensagem_bytes).to_bytes(4, 'big'))
        envia.send(mensagem_bytes)
```

Essa função ela vai ser usada para fazer o caminho contrario ao que o arquivo fez, tem como objetivo propagar o resultado da execução dos arquivos.

Etapas da função:

1. Se conecta com quem vai receber a mensagem.
2. Envia o tamanho da mensagem e após, envia a mensagem em si.

- Receber String

```
def receber_resultado():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as recebe:
        recebe.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        recebe.bind((IP_PORTA_OndeClienteOuve[0], IP_PORTA_OndeClienteOuve[1]))
        recebe.listen(1)
        resultado = ""

        conn, addr = recebe.accept()
        with conn:
            while True:
                dados = conn.recv(1024)
                if not dados:
                    break
                resultado += dados.decode('utf-8')

    return resultado
```

Essa função tem como objetivo receber o resultado da execução que esta sendo enviado de volta para o cliente.

Etapas da função:

1. Espera até ouvir alguma mensagem
2. Recebe 1024 bytes de mensagem e concatena em uma variável resultado até que não se tenha mais para receber.
3. Por fim, retorna a lista resultado.

2.3 Retransmissão das Mensagens

A nossa retransmissão é tratada de duas maneira diferente, uma para o envio do arquivo que vai ser compilado e uma para o retorno do resultado da execução.

Envio do Arquivo

Envio de Arquivos do Cliente para o Portal

- **Segmentação do Arquivo:** O arquivo que será enviado pelo cliente é segmentado em pedaços menores. Essa segmentação facilita a transmissão dos dados através do socket.
- **Recepção dos Pedaços pelo Portal:** O portal recebe esses pedaços de dados um a um. Cada pedaço recebido é adicionado a um novo arquivo que está sendo criado no diretório do portal.
- **Construção do Arquivo no Portal:** À medida que os pedaços vão chegando, o portal continua a adicionar esses pedaços ao arquivo que está sendo construído no seu diretório. Isso continua até que todos os pedaços do arquivo original tenham sido recebidos.

Envio de Arquivos do Portal para o Servidor

- **Segmentação do Arquivo:** O arquivo completo que foi montado no portal é, novamente, segmentado em pedaços menores para ser enviado ao servidor.
- **Recepção dos Pedaços pelo Servidor:** O servidor recebe os pedaços do arquivo enviados pelo portal. Cada pedaço recebido é adicionado a um novo arquivo que está sendo criado no diretório do servidor.
- **Construção do Arquivo no Servidor:** À medida que os pedaços vão chegando, o servidor continua a adicionar esses pedaços ao arquivo que está sendo construído no seu diretório. Isso continua até que todos os pedaços do arquivo original tenham sido recebidos.

Envio do Resultado

Execução dos Arquivos no Servidor

- **Compilação e Execução:** O servidor compila e executa o arquivo recebido. Após a execução, o servidor obtém o resultado da execução (saída do programa, mensagens de erro, etc.).

Envio dos Resultados do Servidor para o Portal:

- **Envio do Tamanho da Mensagem:** Antes de enviar o resultado da execução, o servidor envia o tamanho da mensagem de resultado para o portal.
- **Envio da Mensagem de Resultado:** Após enviar o tamanho, o servidor envia a mensagem de resultado inteira para o portal.

Concatenando Resultados no Portal:

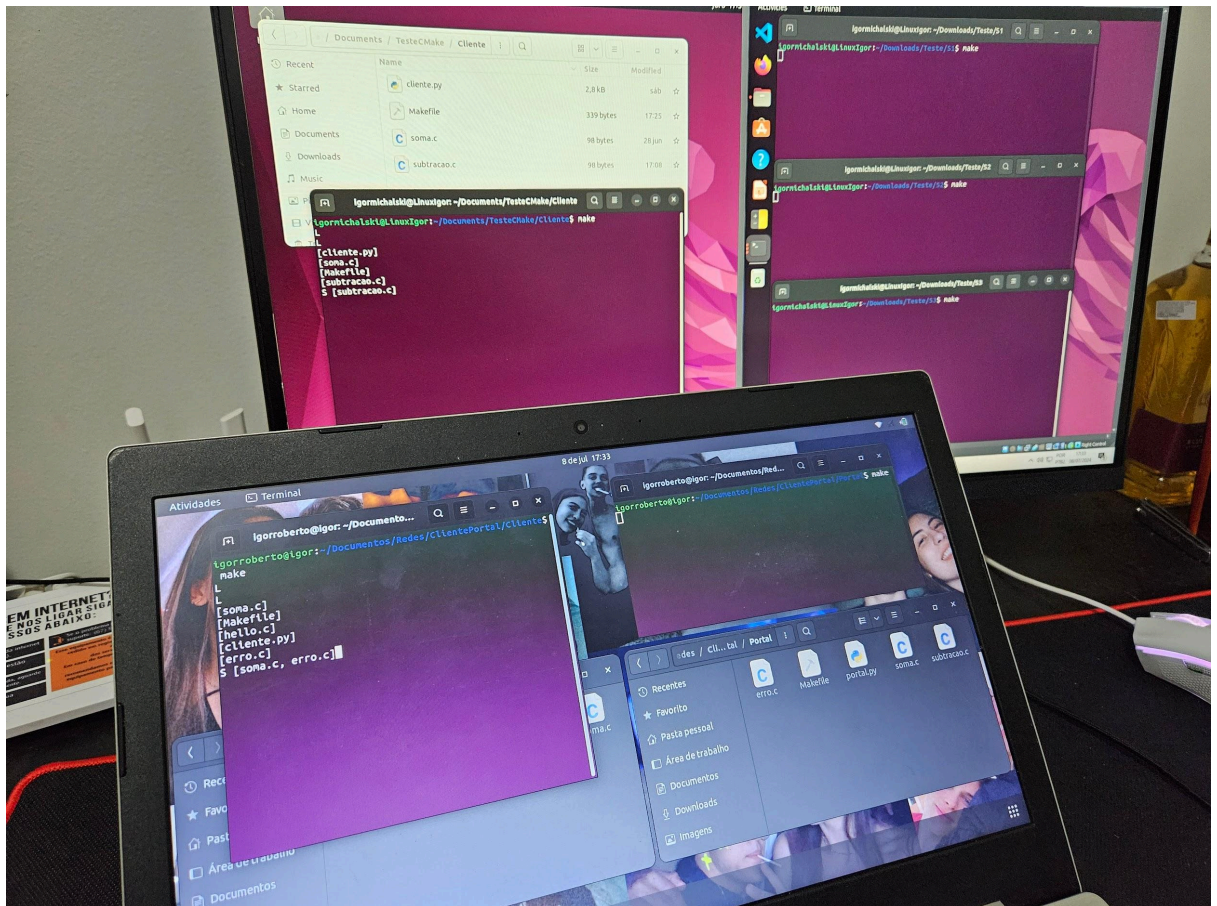
- **Recepção do Tamanho da Mensagem:** O portal primeiro recebe o tamanho da mensagem de resultado enviada pelo servidor.
- **Recepção da Mensagem de Resultado:** Em seguida, o portal recebe a mensagem de resultado inteira com base no tamanho informado.
- **Concatenando Resultados:** Se o cliente tiver enviado mais de um arquivo, o portal vai concatenando os resultados de todas as execuções em uma variável. Esta variável armazena os resultados conforme chegam.
- **Formação do Resultado Completo:** Quando todos os resultados das execuções estiverem completos, o portal terá uma variável contendo o resultado de todas as execuções de arquivos enviados pelo cliente.

Envio do Resultado Completo para o Cliente:

- **Envio do Tamanho da Mensagem:** O portal envia primeiro o tamanho da mensagem com todos os resultados concatenados para o cliente.
- **Envio da Mensagem Completa:** Em seguida, o portal envia a mensagem completa com todos os resultados concatenados.
- **Recepção e Exibição pelo Cliente:** O cliente recebe o tamanho da mensagem e depois a mensagem completa, exibindo na tela o resultado das execuções de todos os arquivos enviados.

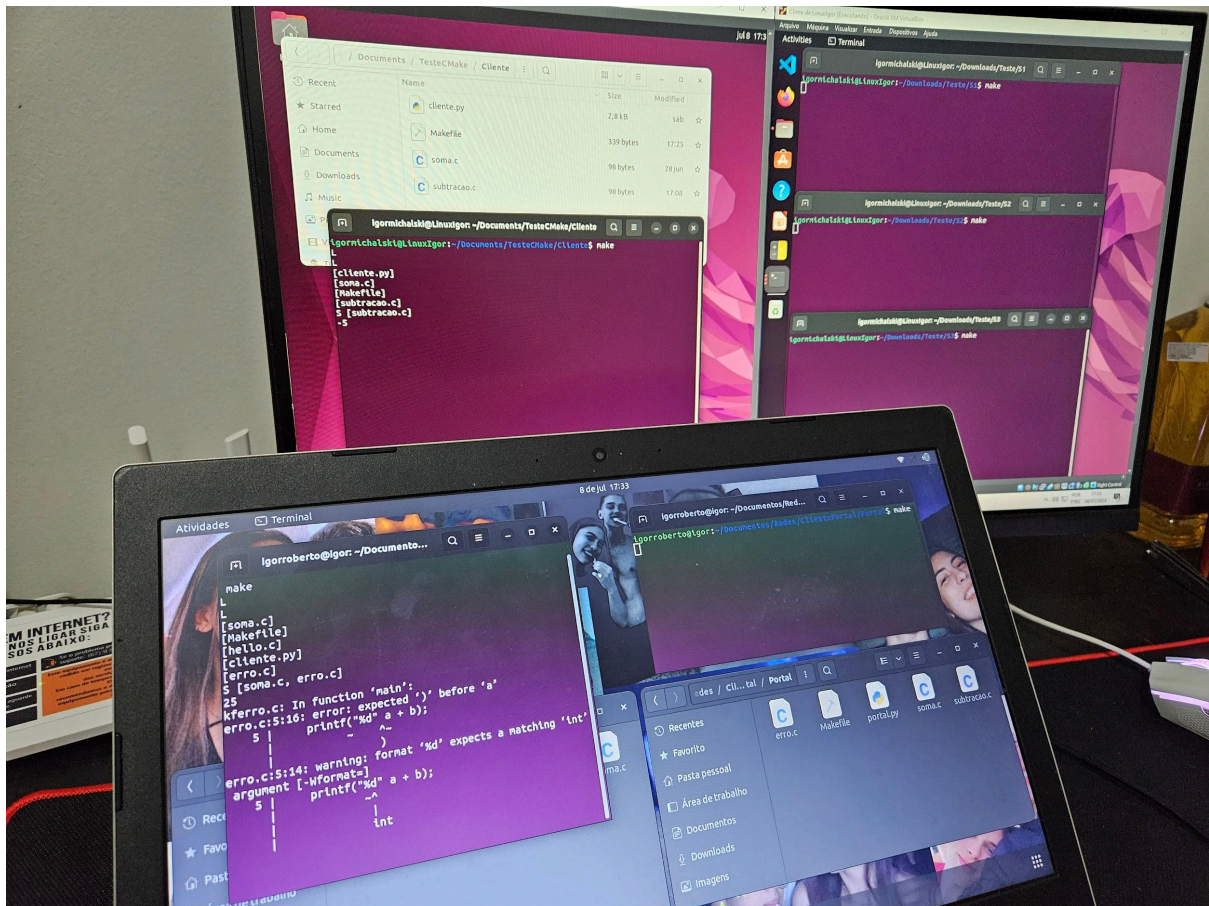
2.4 Testes e Analise

Aqui segue prints do teste realizado com 2 clientes, 1 porta e 3 servidores.



Aqui neste caso o cliente 1 esta enviando o subtracao.c e o cliente 2 esta enviando os arquivos soma.c e o arquivo erro.c.

O código tem como objetivo retornar os resultados e no arquivos erro.c especificamente vai retornar um erro de compilação, pois nele contem um erro.



Aqui vemos que no cliente 1 ele retornou -5 que era o resultado esperado e no cliente 2 retornou 25 e um erro de compilação, também como esperado.

REFERÊNCIAS

BÓSON TREINAMENTOS. Programação de Redes com Python - Introdução ao Módulo socket. Disponível em: <<https://www.youtube.com/watch?v=v3zVT4HJxYI&t=2s>>. Acesso em: 3 jul. 2024.

HOWTO sobre a Programação de Soquetes — documentação Python 3.8.9. Disponível em: <<https://docs.python.org/pt-br/3.8/howto/sockets.html>>.

PYTHON. socket — Low-level networking interface — Python 3.8.1 documentation. Disponível em: <<https://docs.python.org/3/library/socket.html>>.

send() function of python socket class | Pythontic.com. Disponível em: <<https://pythontic.com/modules/socket/send>>.

SIMPLIFICANDO REDES. Python – Enviando arquivos entre duas máquinas. Disponível em: <<https://www.youtube.com/watch?v=H6oyVjmglBo&t=1s>>. Acesso em: 3 jul. 2024.

Scripts Cliente e Servidor com Módulo Socket em Python - Exemplo. Disponível em: <https://www.youtube.com/watch?v=vbUuJ2_6wqs>.